The `.aamt` Substrate: Memory-Mapped Hexadeca-Tree Storage and Zero-Copy Geometric Routing (the Micro-VM Layer)

Authors: Weslyn Cory Whitehead Jr.[1]

Affiliations: [1] AsAManThinks Research, Berkeley, CA, USA

Corresponding author: weslyn@asamanthinks.com

ORCID: https://orcid.org/0009-0005-7707-3210

Submitted: July 2026 **Working Paper Series:** AAMT-WP-23 **Anchor DOI:** 10.5281/zenodo.19600795

Status: Working paper, not peer reviewed. The reference implementation is `packages/libaamt` — a dependency-free C++17 core with a plain C ABI (`include/aamt/aamt.h`), a pack/inspect/query/bench CLI (`tools/aamt_cli.cpp`), and a deterministic round-trip test suite (`tests/roundtrip.cpp`). Every quantitative claim in Section 6 is produced by `aamt bench` and `examples/bench_cold_start.py` and is reproducible on commodity Apple Silicon hardware. This paper supersedes the earlier WP-23 draft ("Micro-VM Runtime Environment"); Section 8 records what the draft claimed, what measurement and prior AAMT results required us to correct, and why the corrected design is stronger.

Abstract

The AAMT series has produced three runnable memory systems: topologically protected trit anchors (EXP-01), voxel-steered output eviction over a TERA/HNSW index (WP-21), and Odu coverage maps for loop escape (WP-22). Each currently persists through a different ad-hoc path — TSV, JSON, Python dictionaries, and an HNSW sidecar — so every session pays a full parse-and-rebuild cost, and no two surfaces of the platform (desktop Python, browser visualizer, Unity) can open the same artifact. This paper specifies the **`.aamt` substrate**: a single memory-mapped binary format holding a hexadeca-tree spatial index over the 4D TERA cube, with 64-byte cache-line nodes, popcount-indexed sparse children, a zero-copy payload arena, and a deterministic sidecar journal that makes every query bit-exactly replayable. The load-bearing structural result is the **orthant correspondence**: a binary split of 4D TERA space yields exactly $2^4 = 16$ children per node — the 16 Meji — and two tree levels yield exactly $16 \times 16 = 256$ cells — the 256 Odu. The Meji/Odu lattice is therefore not a naming convention layered on the index; it *is* the unique natural spatial index of TERA space, and a query's routing path is literally an Odu word. We position the substrate honestly: it is the geometric memory layer that runs *beside* a conventional LLM runtime (the architecture WP-21 already established), not a replacement for transformer weights, and its routing cost is $O(\text{depth})$ — bounded and scan-free — not $O(1)$. The substrate's first production integration (Section 9, `memory_index_bridge.py`) then falsified its own most obvious application: reconstructing a bulk id/label table through the substrate loses to `json.load` by an order of magnitude, because per-record ctypes marshalling overhead dominates the native tree walk's advantage. What shipped instead — labels stay in JSON; the substrate serves a new O(depth) coarse-routing command alongside HNSW's fine discrimination — is the correction, and it is the more interesting result: the right division of labor was not the one first assumed, and measurement found it in under an hour rather than in production. Section 10 adds a redundancy scheme built from a single exact geometric operation — a true Householder reflection of the TERA cube's T axis, distinguished from the antipodal map (which, in this even dimension, is orientation-preserving and so not actually a mirror) — giving every record a physically separate redundant copy and a gate, `verifyChirality`, that catches "bad reflections" by re-deriving the relationship rather than trusting stored state; verified against real, injected byte-level corruption, not simulated failure.

1. One file, three papers' worth of runtime

The platform's geometric memory is fragmented across formats that were each correct for the experiment that produced them:

Producer	Artifact today	Cost paid per session
EXP-01 trit anchors	lattice state in JS worker memory	recompute
WP-20/21 voxel memory	TSV + JSON + HNSW sidecar via Python bridges	full parse + index rebuild
WP-22 coverage maps	Python dict in `odu_coverage_map.py`	recompute
`@aamt/aamt-foundations`	JSON consumed by Python/TS/C#	parse per consumer

The `.aamt` substrate replaces the *persistence* of all four with one artifact and one operation: `mmap`. A memory-mapped file has no deserialization step — the on-disk bytes are the in-memory data structure. Opening a multi-gigabyte substrate costs one system call plus page faults for only the regions actually touched. The same bytes are readable by:

- **Python** (desktop Alchemist) via `ctypes` against the C ABI,
- **the browser** (Chiral Lattice visualizer) via an Emscripten/WASM build,
- **C# / Unity** (DreamOS, Streetlife) via P/Invoke,
- **native C++** (the Micro-VM core itself).

This is the Dual-Stack model of the original draft, kept: a thin native core under whatever expression layer each surface already uses. What is dropped from the draft is the introduction of a new front-end language; the C ABI + WASM boundary serves every existing stack without adding one.

2. The orthant correspondence

Let a node own an axis-aligned box $B \subset [0,1]^4$ in TERA space. Splitting B at its midpoint on all four axes produces $2^4 = 16$ orthants. Assign each orthant the index

$$m \;=\; b_T + 2\,b_E + 4\,b_R + 8\,b_A, \qquad b_d = [\,v_d \geq \mathrm{mid}_d\,] \in \{0,1\},$$

i.e. one bit per TERA dimension. The 16 child indices **are the 16 Meji**, under the same bit convention `@aamt/vortex-runtime` already uses for `mejiDistribution`. Descending two levels assigns a pair (m_1, m_2), i.e. one of $16 \times 16 = 256$ cells — **the 256 Odu**. In general, the routing path of a query to depth k is a word $m_1 m_2 \cdots m_k$ in the Meji alphabet: *every query has an Odu-word address, and the tree is the address book.*

Two consequences make this correspondence load-bearing rather than decorative:

2.1 The tree is the hard limit of the existing soft projection. `vortex-runtime` computes a product-Bernoulli distribution over Meji, $p(M_m) = \prod_d [v_d]^{b_d}\,[1-v_d]^{1-b_d}$ (with v_d the TERA component). The hard orthant split above is exactly $\arg\max_m p(M_m)$ at $\mathrm{mid}_d = \frac{1}{2}$. Tree descent is therefore the discrete limit of the projection the platform already ships; soft routing (Section 7) recovers the full distribution when needed.

2.2 The Product Fisher–Rao Metric has a closed form here. For product-Bernoulli distributions the Fisher–Rao distance separates per dimension:

$$d_{FR}(u,v) \;=\; 2 \sum_{d \in \{T,E,R,A\}} \left|\, \arcsin\sqrt{u_d} - \arcsin\sqrt{v_d}\,\right|.$$

This is the "geometric path routing" metric of the original draft, now with a definition, a formula, and an existing in-house implementation surface (WP-22 uses the same metric for escape steering). It costs eight `asin` calls — there is nothing exotic to build.

2.3 The chiral tie-break O_x has one precise job. When a coordinate lands *exactly* on a splitting hyperplane ($v_d = \mathrm{mid}_d$), the comparison $v_d \geq \mathrm{mid}_d$ resolves it upward. This is a right-continuous choice — the one-sided limit 0^+, the same signed-zero object EXP-01 identified — and it is the entire, exact role of O_x in the router: a deterministic, orientation-bearing decision at a point of perfect symmetry. Nothing more is claimed for it here.

3. Binary specification

3.1 File layout

```
+---------------+--------------------------+--------------------------+
|  Header (128B) |  Node arena (64B × N)     |  Payload arena (records)  |
+---------------+--------------------------+--------------------------+
      mmap'd read-only; content-hashed; immutable after pack
```

The file is immutable after packing and is always mapped `PROT_READ`. Anything that must grow at runtime — above all the lineage journal — lives in a sidecar (Section 5). This resolves a contradiction in the earlier draft, which mapped the file read-only and also committed ledger writes into it.

3.2 The 64-byte node, corrected

The draft's node held `uint64_t children_offsets[16]` (128 bytes) and claimed a 64-byte cache-line footprint; the fields as specified total 156 bytes and pad to **192 bytes — three cache lines**. Worse, a dense 16-slot child table charges every sparse region for children it does not have.

The corrected node borrows the occupancy-bitmask technique from Hash Array Mapped Tries and Efficient Sparse Voxel Octrees. A 16-bit mask records which of the 16 Meji children exist; existing children are packed contiguously; the slot of child m is recovered with one instruction:

```
slot(m) = popcount(child_mask & ((1 << m) - 1))
child   = node_arena[children_base + slot(m)]
```

```
struct alignas(64) AamtNode {
    uint32_t coordinate_id;   // packed Meji path of this node (8 nibbles max)
    float    chirality_spin;  // reserved for field-state annotation (EXP-01)
    uint16_t child_mask;      // occupancy over the 16 Meji orthants
    uint16_t flags;           // bit 0: leaf; bits 8..15: depth
    uint32_t record_count;    // records in this leaf's bucket
    uint64_t children_base;   // node-arena index of first child (contiguous)
    uint64_t payload_offset;  // byte offset into payload arena
    uint64_t payload_size;    // byte length of this leaf's bucket
    uint8_t  reserved[24];
};
static_assert(sizeof(AamtNode) == 64);
```

$4 + 4 + 2 + 2 + 4 + 8 + 8 + 8 = 40$ bytes of live fields, 24 reserved — a true single cache line, verified by `static_assert` in the reference implementation. Empty orthants cost zero bytes. `coordinate_id` packs the node's own Meji path, so every node carries its Odu-word address in its first field.

3.3 Payload records

Leaf buckets are contiguous runs in the payload arena:

```
Record := { uint64 external_id; float tera[4]; uint32 size; uint8 data[size]; pad to 8B }
```

`data` is opaque to the router: an embedding shard, a WP-21 evicted capsule, an EXP-01 trit word, an HNSW neighborhood, a foundations entry. The router selects; it never interprets. Readers receive **pointers into the mapping** — no copy occurs between disk format and consumer.

3.4 Header

128 bytes: magic `"AAMT"`, format version, node count, record count, arena offsets/sizes, tree parameters (max depth, bucket capacity), and an FNV-1a content hash over both arenas. The hash is the file's identity in the journal.

4. Execution model: the CPU routes, the accelerator computes

The draft's asymmetric task allocation survives review intact, with one platform correction.

- **The Navigator (CPU).** Tree descent is branchy pointer-chasing over 64-byte nodes — the access pattern modern branch predictors and L1 caches are built for, and the pattern that destroys GPU wavefront coherence. Descent to depth 8 touches ≤ 512 bytes of node data.
- **The Engine (GPU/SIMD).** Payload math (similarity over selected embedding shards, batch transforms) is uniform dense arithmetic over the records the CPU selected.

Apple Silicon (true zero-copy, measured). Unified memory makes the handoff literal: `MTLBuffer newBufferWithBytesNoCopy:` wraps page-aligned regions of the mapping so Metal shaders read the *same physical pages* the CPU routed to. The payload arena is page-aligned in the format for exactly this reason. The reference implementation (`packages/libaamt/metal`) runs an L2-norm compute kernel — compiled at process startup from an embedded MSL string via `newLibraryWithSource`, so no `.metal` file or Xcode shader compiler is required, only `Metal.framework` — over a uniform-stride payload arena, twice: once wrapped zero-copy, once via the copying `newBufferWithBytes:`. Measured on an Apple M1 Max, identical data, identical kernel:

arena size	zero-copy	copy	speedup
4.8 MB (50k × 16 floats)	2.56 ms	1.44 ms	**0.56**× (copy wins)
57.6 MB (200k × 64 floats)	6.96 ms	14.59 ms	**2.10**×
528 MB (500k × 256 floats)	30.10 ms	125.70 ms	**4.18**×

Correctness is checked against a CPU-recomputed reference for every record (max error $\approx$ 1e-5, float precision). The honest reading: zero-copy wins, and wins by a growing margin, once the arena is large enough that the copy's cost dominates; below roughly the tens-of-MB range here, `newBufferWithBytesNoCopy` wrapping overhead (page-boundary handling, buffer registration) can exceed the cost of simply copying a small region. The crossover is measured, not assumed — a working demonstration of the payload arena's zero-copy design earning its keep, and of exactly where it does not yet.

Linux. `open` + `mmap(PROT_READ, MAP_SHARED)` as in the draft, plus `madvise(MADV_WILLNEED)` issued for a child subtree at descent time — prefetching stated honestly as a hint to the page cache, not "elimination of page faults."

Web/WASM (correction). WebGPU has **no shared address space** with WASM linear memory; a raw pointer cannot be handed to a browser compute shader. The web target therefore uses an explicit staging step: the WASM-side router selects records, and only those bytes are uploaded via `writeBuffer`. Because the router's whole purpose is to select a small working set, the staging copy is proportional to the answer, not to the file.

5. The lineage journal and deterministic replay

Every query appends one fixed-width record to a sidecar `.aamtj` file:

```
JournalRecord := { uint64 t_ns; uint64 file_hash; float tera[4];
                   uint32 path;  // packed Meji nibbles - the query's Odu word
                   uint32 depth }
```

Because the substrate is immutable and content-hashed, `(file_hash, tera)` determines the routing outcome completely; the journal is a total, replayable record of every routing decision the system ever made, at 40 bytes per query. Replay is a test, not a promise: re-running the journal against the file must reproduce every path bit-for-bit. This carries EXP-01's seeded-determinism ethos into the runtime layer. We claim *deterministic replay of internal state* — a falsifiable property — and nothing beyond it.

6. What is measured

The reference implementation ships two benchmarks. First measured results (Apple M1 Max, 32 GB unified memory, single-threaded, July 2026; seeds fixed, reproducible via `aamt bench` and `examples/bench_cold_start.py`):

1. **Cold start** (100,000 records, 6.9 MB substrate). Current-path Python — parse TSV into dicts, as the desktop bridges do today — reaches its first usable lookup in **64.8 ms** and grows linearly with N.

`aamt_open` + one routed query via `ctypes` takes **1.2 ms** (dominated by `dlopen`/`ctypes` setup) and is independent of N: a **54× cold-start speedup** at this size, unbounded as N grows. Native `aamt_open` alone — mmap + header validation — measures **0.021 ms on a 1,000,000-record file**.

2. **Routing latency** (1,000,000 records; 69,962 nodes × 64 B ≈ 4.5 MB index; packed in 434 ms). Uniform random queries through the full C ABI: **397 ns/query — 2.52 M queries/s** single-threaded, including per-call record-view marshalling (a known v0 overhead: the C bridge allocates a scratch vector per query; routing alone is cheaper). An example route: query $(0.72, 0.31, 0.88, 0.15)$ resolves to Odu word `5.6.13.1` at depth 4 and returns a 20-record bucket whose members are visibly TERA-near the query point — the tree is doing coarse geometric retrieval exactly as WP-20's division of labor prescribes.
3. **Round-trip integrity** (`tests/roundtrip.cpp`): 20,000 seeded records packed, every record retrieved back byte-exact by routing its own coordinate, and 1,000 journaled queries replayed **bit-for-bit** against the content hash. Passing in CI is the falsifiable form of Section 5's claim.
4. **Cross-platform identity** (`packages/libaamt/wasm`): the identical `core.cpp`/`aamt_c.cpp`, recompiled unmodified via Emscripten, opens a `.aamt` file packed by the native CLI inside a WASM sandbox and returns byte-identical records for the same query (`wasm/smoke_test.mjs`, `wasm/smoke_test_client.mjs`) — the "one file, every surface" claim of Section 1, checked rather than asserted.
5. **Apple Silicon zero-copy** (`packages/libaamt/metal`, Section 3.2): see the measured zero-copy-vs-copy table above.

Claims not made: no $O(1)$ (descent is $O(\text{depth}) = O(\log N)$; the defensible statement is *scan-free and bounded* — cost is independent of how much memory is stored beyond its logarithm); no general GPU throughput claim beyond the specific L2-norm kernel measured in Section 3.2 — a batch similarity/transform kernel over real embedding payloads is the natural next measurement, not yet taken.

7. Positioning: beside the model, not instead of it

The earlier draft framed this layer as executing "topological model structures" in place of transformer weights. Two in-house results forbid that framing:

- **WP-20 measured pure-TERA recall@10 at 0.017** and correctly concluded that TERA geometry is coarse routing while HNSW/embedding residuals carry fine discrimination. The substrate honors that division of labor: the tree routes coarsely; payloads carry the discriminative structures.
- Dense transformer inference touches essentially all weights per token; there is no spatial sparsity for a tree to exploit, and mmap'd weight files already exist (GGUF). Rebuilding LLM inference would be a multi-year detour to tie a race llama.cpp already runs.

The substrate is therefore the **memory organ** of the architecture WP-21 already established: the LLM generates on its existing runtime; the substrate receives evicted geometry, serves coverage state (WP-22), anchors trit words (EXP-01), and answers routed retrieval — locally, deterministically, and identically on every platform surface.

Soft routing, implemented — and rescoped. Section 2.1 noted that hard routing (`mejiIndex`, argmax) is the discrete limit of `mejiDistribution`'s full product-Bernoulli weight over all 16 root orthants. `routeSoftTopK` (`aamt_query_soft` in the C ABI) makes the full distribution queryable directly: it ranks all 16 root-level Meji orthants by `mejiWeight` — descending — and hard-descends the top k each to its own leaf, reporting *empty* orthants too (weight, zero records). Wired into `memory_index_bridge.py`'s `route` command (`soft: true`); weights are checked to sum to 1 and sort descending in `tests/roundtrip.cpp`, and the rank-0 branch is checked against `route()`'s hard result across 200 random queries — it must agree exactly, since a soft ranking's top pick and a hard argmax are the same computation by construction.

An earlier version of this section claimed soft routing as "exactly WP-22's escape-steering need." That is wrong, and the error is worth recording precisely rather than quietly fixing: `routeSoftTopK`'s weight is a function of the query coordinate *alone* — how strongly it belongs to each orthant geometrically. WP-22's `OduCoverageMap.probe()` (`odu_coverage_map.py`) answers a different question — which cell is under-visited *this session* — from a 256-cell running occupancy count with no relationship to the substrate's persisted id/tera table at all. A "coldest" soft-routing branch is the orthant a query geometrically *resembles*

least; it has no notion of visit history and cannot stand in for one. Swapping it into `probe()` would have silently discarded the session's actual coverage signal.

What soft routing *is* verified to do (`examples/` sparse-cluster tests, not yet promoted to a committed benchmark): near real data, it widens the candidate pool past a single hard bucket's boundary luck — a query at $(0.49, 0.51, 0.49, 0.51)$ against a substrate with data spread over $[0.3, 0.7]^4$ hard-routes to a 2-record bucket, while its rank-1 soft branch alone holds 6. Query from a region genuinely far from all data and every branch correctly reports nothing nearby — geometric similarity to a query that has no similar data is not a bug to route around. This is real, tested, and useful (candidate widening for `query`/`route`, and any future consumer that wants "near neighbors of this coordinate" without walking HNSW) — it is just not WP-22's escape search, and the two should not be merged.

What actually closes the WP-22 gap: the persisted cell profile. The misidentification above exposed the real hole in Section 1's "serves coverage state (WP-22)" claim: `OduCoverageMap` received *nothing* from the substrate. Its escape search targeted abstract geometric cell centers for any cell not visited in the current session — even when the substrate held persisted records that know exactly what lives there. The repair respects the division of labor instead of merging the two systems:

- **The substrate contributes what it uniquely knows** — `aamt_odu_profile` walks every persisted record once and returns per-Odu-cell counts and mean TERA centroids (256 cells; the cross-session answer to "what actually lives in cell j"). Correctness is checked against the router itself: every record's profile cell must equal the top-2 nibbles of its own routing path (`tests/roundtrip.cpp`).
- **The session keeps what is session's** — `OduCoverageMap.seed_persistent` consumes the profile as the escape search's *fallback target tier* (session centroid → persisted centroid → geometric center) and deliberately does **not** seed occupancy counts: coverage and entropy are statements about *this session's* trajectory, and pre-loading counts would fake coverage the session never earned. The prior refines *where* an escape steers, never *whether* one triggers.
- **The conventions are bridged explicitly, not assumed** — the substrate's cell index is its routing path (Meji-major, $(m_1 \ll 4)|m_2$, T at bit 0); `voxel.py`/OCM pack axis-major with T in the top bits. The permutation (`tree_cell_to_axis_major`) is verified against `voxel.tera_to_odu` across 20,000+ coordinates including every quartile boundary — the two quantizers agree everywhere, including the O_x (`>=`) boundary rule, so only the bit packing differs.

The end-to-end proof (`examples/test_escape_seed.py`, run against the real bridge wire protocol and the real `OduCoverageMap`): a hot cell at $(0.74, 0.6, 0.6, 0.6)$ escapes to the adjacent cell in both arms, but the seeded arm targets the persisted centroid $(0.765, 0.599, 0.600, 0.601)$ — where the previous session's memory actually sits — while the unseeded control targets the abstract center $(0.875, 0.625, 0.625, 0.625)$. A distance control confirms persisted data far from the hot point does *not* hijack the escape: Fisher–Rao proximity still decides *which* cell wins; persistence only improves the *target within it*. Opt-in from the WP-21 loop via `WP21_SUBSTRATE_SEED=<path.aamt>`.

That check surfaced a real edge case, corrected before shipping: at the TERA cube's exact center, all 16 orthants weigh precisely 1/16 — a tie `mejiIndex` resolves deterministically via its `>=` rule (the O_x chiral tie-break) but that a naive weight-sort does not, since sorting by float equality has no opinion about *which* equal element goes first. The fix breaks ties in favor of whichever orthant `mejiIndex` would actually pick, so `routeSoftTopK`'s rank-0 branch never silently disagrees with `route()`'s hard result — a regression test at the exact center (`tests/roundtrip.cpp`) locks this in.

8. Corrections ledger

In the EXP-01 tradition of recording predictions falsified by measurement, the changes from the first WP-23 draft:

1. **Node layout arithmetic falsified the cache-line claim** (156 B → 192 B padded, three lines). Corrected via bitmask + popcount children; the claim is now true and statically asserted.
2. $O(1)$ **routing** replaced by $O(\text{depth})$, stated as scan-free.
3. **WebGPU raw-pointer handoff is impossible**; replaced by selective staging.
4. **In-file ledger commits contradicted the read-only mapping**; journal moved to a sidecar keyed by content hash.

5. **Weight-replacement scope withdrawn** — it contradicted WP-20's own measurement; scope re-anchored to the geometric memory substrate.
6. **New front-end stack (Haxe) withdrawn** from the critical path; the C ABI + WASM boundary serves the three stacks the platform already runs. Nothing prevents a Haxe consumer later; nothing requires one now.
7. **"Consciousness" language withdrawn** in favor of the testable property it gestured at: deterministic replay of internal routing state.
8. **Bulk id/label reconstruction assumed a substrate win; measured, it is not** (Section 9). Moving `memory_index_bridge.py`'s label table into `.aamt` and reconstructing it via a ctypes marshalling loop on load costs ~110 ms for 200k short strings against `json.load`'s ~7 ms — a ~15–40x loss — even though the underlying native tree walk alone is ~2.6 ms. The per-record Python/ctypes FFI-crossing tax dominates the native routing advantage for this specific workload. Labels stay in JSON; the substrate is used for what it measures well at instead (point routing).
   Follow-up. The per-record cost traced to two things: constructing a `ctypes.Structure` object for every record (`aamt_all_records`'s array of `aamt_record_view`), and calling `ctypes.string_at()` once per record. A columnar export (`aamt_all_records_columnar`) removes both: one native call fills flat primitive arrays (ids, offsets, sizes — no per-record struct), and one bulk copy pulls every payload into a single Python `bytes` object, leaving only pure bytes-slicing per record with no further FFI crossings. Measured at 200,000 records: **150.6 ms → 53.1 ms, a 2.84x improvement** over the naive loop. It does not overturn the original result — `json.load` still finishes in **10.1 ms**, ~5.25x faster than the improved columnar path, because CPython's JSON parser has no per-element Python bytecode loop at all, while even a bare bytes-slice-and-decode comprehension does. `memory_index_bridge.py`'s design is unchanged: labels stay in JSON. The columnar export is kept as the substrate's primitive for bulk reads regardless, because the comparison that matters is not "columnar vs. `json.load`" for consumers who have no `json.load` — Unity's C# bindings and the WASM build get the 2.84x-faster version from the start, since neither has a competing Python-specific advantage to lose to.
9. **The WASM build's growable memory broke on real-browser testing** — caught live, in Safari, minutes after the first production deploy, not by any automated test in this paper. `-sALLOW_MEMORY_GROWTH=1` makes `WebAssembly.Memory.buffer` a "resizable" `ArrayBuffer`; Safari's `TextDecoder.decode()` rejects resizable buffers outright, including on views taken from Emscripten's own generated glue (reading a UTF8 C string), not just the hand-written JS wrapper's own decode calls — so patching the wrapper alone would not have fully fixed it. The build now uses a fixed 128 MB `-sINITIAL_MEMORY` with growth disabled, which eliminates the resizable-buffer code path entirely rather than chasing every internal `decode()` call site. `HEAPU8.buffer.resizable === false` is now asserted in the WASM smoke tests. Node's V8 never exhibited this — the automated test suite runs under Node, and would not have caught it without the live-browser report.
10. **Soft routing's rank-0 branch could silently disagree with the hard route** at exact weight ties (Section 7). All 16 root orthants weigh precisely 1/16 at the TERA cube's center; `mejiIndex`'s `>=` rule (the O_x tie-break) resolves that deterministically, but a plain weight-descending sort has no such opinion among equal floats. Fixed by breaking ties in `routeSoftTopK` in favor of whichever orthant `mejiIndex` actually picks. Caught by the test written specifically to guarantee the two methods agree — the check that would have made the bug's absence a real claim instead caught the bug it was written to rule out, which is exactly what it was for.

What survived unchanged: the Dual-Stack separation, the hexadeca-tree over TERA space, the CPU-routes/GPU-computes split, unified-memory zero-copy on Apple Silicon, `mmap` as the ingestion primitive, and O_x as the deterministic symmetry-breaker — each now with its claims sized to what the reference implementation demonstrates.

9. First integration: `memory_index_bridge.py`

The Alchemist's cross-session HNSW index bridge (`apps/desktop/scripts/ memory_index_bridge.py`) is the first production consumer of the substrate, and it did not go the way Section 1 anticipated.

What was tried. The bridge already discarded a TERA coordinate for every evicted chunk — `wp21_rolling_output_prototype.py` computes `tera` via the WP-22 lexicon projector for the OCM gate,

then never uses it again. The obvious integration: move the id/label table (`labels: list[str]`, previously a flat array inside `.meta.json`) into a `.aamt` substrate, and reconstruct it on `load()` via `aamt_all_records` + a Python ctypes loop — mmap instead of `json.load`, the direct application of Section 6's cold-start result.

What was measured (`examples/bench_memory_index_bridge.py`, 20,000 items, real subprocess wire protocol): the substrate reconstruction path costs **10.02 ms**; `json.load` costs **0.88 ms** — json wins by **11.4x**, worsening to ~15–40x at 200k items. The native tree walk underneath the substrate path is ~2.6 ms at that scale (measured separately); the remaining ~7.4 ms is entirely the cost of 20,000 individual `ctypes.string_at()` calls and Python object construction. CPython's C-implemented JSON parser has no such per-element FFI boundary to cross and simply wins at "materialize N short strings."

What shipped instead. Labels stay in `.meta.json`, unchanged from before this integration. The substrate is still packed on every `save()` — now serving a genuinely new wire command, `route`, that performs the coarse TERA-space lookup Section 7 describes directly against persisted memory, with no HNSW graph involved:

```
route: {"cmd":"route","args":{"tera":[T,E,R,A],"k":8}}
  -> {"odu_word":"13.3.10","depth":3,"bucket_size":3,
     "candidates":[{"id":"mem-...","tera":[...]}, ...]}
```

Measured at 20,000 items: **0.080 ms** per `route` call, available immediately after `save()` with no reload required, and correctly returning the queried item within its own bucket (verified in the same benchmark). This is the honest shape of the integration: `query` (HNSW, fine discrimination) and `route` (substrate, coarse geometric lookup) now sit side by side, doing the jobs each was independently measured to be good at — the WP-20 division of labor, realized as two wire commands instead of one being quietly worse than the tool it was meant to replace.

---

10. Mirror pairing: geometric redundancy from an exact reflection

Every mechanism above answers "where is a coordinate" or "what lives near it." None answers "is what I just read actually intact." This section adds that: a redundancy scheme built from a single, exact geometric operation rather than a bolted-on checksum, plus the gate that operation makes possible for free.

10.1 The reflection

Define $H(v) = (1 - v_T,\ v_E,\ v_R,\ v_A)$ — flip the T axis about the cube's own midpoint. Two structural facts, both load-bearing and both tested (`tests/roundtrip.cpp`):

- H **is an exact involution**, $H(H(v)) = v$, with no stored inverse and no numerical drift — `1 - (1 - x)` for the range of floats a packed coordinate takes is exact in every case this paper's tests exercise.
- H **is a true reflection, not the antipodal map.** Flipping all four axes ($v \mapsto 1-v$) reads like the more natural "opposite point," but in this *even* dimension it is orientation-**preserving** ($\det = (-1)^4 = +1$) — a rotation wearing a mirror's clothes. Flipping a single axis is orientation-**reversing** ($\det = -1$): the genuine article. The distinction is exactly WP-23 §2.3's Ox tie-break territory — parity, not just position, decides whether an operation is a mirror.

A corollary worth stating precisely because it is *almost* true and the exception matters: for generic (non-dyadic) coordinates, `route(H(v)).path` equals `route(v).path` with the T-bit of every nibble flipped — H flips exactly the first split's T-bit, and because `childBox` leaves the other three axes' bounds untouched, the flip propagates by induction through every subsequent level. The exception: a coordinate landing exactly on a splitting boundary at some depth (a dyadic rational — measure zero, not absent) can have that level's bit left unchanged for both v and $H(v)$ by the router's own Ox rule. This is not papered over — the verification path below never uses the shortcut, only real re-routing, precisely so a boundary case is a documented curiosity rather than a correctness bug.

10.2 Mirror pairing and the chirality gate

`Builder::addMirrored` inserts a record normally, then inserts a second, full copy — same id, same payload — at the leaf $H(v)$ routes to. This is real 2-times storage for real redundancy: the mirror lands under the opposite T-orthant at the root, a different subtree, a different region of the payload arena, so damage localized to one region does not take out both copies of a record.

`Reader::verifyChirality` is the gate: for every primary, it re-routes $H(v)$ for real (never the shortcut) and confirms a mirror-flagged record with matching id exists there with byte-identical payload. Either failure — mirror missing (a structural break: the primary's own coordinate no longer reflects to where its mirror should be) or payload mismatch (a content break: two independently stored copies disagree) — is what this paper calls a **bad reflection**, counted rather than silently served.

What two copies honestly cannot do. On a pure payload mismatch where both copies are structurally locatable and internally consistent, this detects disagreement but cannot decide *which* copy is corrupt — that needs a third copy or an independent checksum, and this format stores neither. `Reader::recoverRecord` is scoped to match: it only falls back to the mirror on a *structural* failure (no primary found at the hinted coordinate — its identifying bytes are gone, not just its content), where finding a self-consistent mirror at the mathematically predicted location is a real, positive signal, not a guess between two disagreeing copies.

10.3 Verification

`tests/roundtrip.cpp` packs 2,000 mirrored records and checks three things, the last two by editing real on-disk bytes with a separate file handle — not simulated corruption:

1. **Clean substrate:** 2,000 primaries, 2,000 mirrors, 2,000 consistent, zero inconsistent.
2. **Payload-only corruption** (one byte flipped in a primary's payload, its coordinate untouched): `verifyChirality` reports exactly one inconsistency, correctly identifying the corrupted id. `recoverRecord`, called with the correct coordinate, finds the primary structurally present and returns it — corruption and all. This is the honest limit from §10.2, demonstrated rather than hidden: the test asserts the corrupted byte comes back unchanged, because that is what a system with no independent checksum can actually promise.
3. **Identity corruption** (a primary's id field scrambled — its content physically present but no longer findable by id at its own leaf): `recoverRecord`, given the id and coordinate the caller always knew independent of the now-garbled on-disk bytes, falls through to the mirror and recovers the exact original payload and coordinate.

Both corruption tests compute the target record's real file offset from the packed layout (header size, node count, page-aligned payload arena start) and edit the file directly — the same class of test as WP-23 corrections ledger #9's browser bug: catch it by actually doing the thing, not by reasoning that it should work.

11. Redundancy in the substrate's own alphabet: GF(16) Reed–Solomon

Mirror pairing (§10) costs a full second copy to protect against corruption. This section adds a cheaper, complementary mechanism for the common case — a single corrupted symbol — built from the substrate's own alphabet rather than a generic byte-oriented code: Meji is already a 4-bit symbol (one of 16), so `packages/libaamt/src/gf16.hpp` implements GF(16) field arithmetic (log/antilog tables from the primitive polynomial $x^4 + x + 1$) and `rs_gf16.hpp` a systematic RS(15,13) code over it — 13 data nibbles, 2 parity nibbles, $n = 15$ being GF(16)*'s full cyclic order and so the largest block this field supports natively.

What is proven, exhaustively, not sampled. All 225 single-symbol-error cases (15 positions × 15 nonzero delta values) correct to the exact original — every one, not most (`tests/test_rs_gf16.cpp`). Wired into the substrate via `Builder::addWithECC`, and verified against a real on-disk corruption injection at a computed byte offset (`tests/roundtrip.cpp`, mirroring §10's methodology): one nibble flipped in a packed record's payload, decoded back to the exact original with `corrected_symbols == 1`.

What is measured, and states its own ceiling precisely. A code with minimum distance $d = 3$ is textbook-described as "detects up to $d - 1 = 2$ errors, corrects up to $\lfloor (d-1)/2 \rfloor = 1$" — but that line describes the code's distance, not what a *minimum-redundancy* decoder (exactly 2 parity symbols, no

margin beyond what $t = 1$ correction strictly requires) can independently verify. Exhaustive testing over all $\binom{15}{2} \times 15 \times 15 = 23{,}625$ two-simultaneous-error patterns found: **86.7% (20,475 cases) are silently "corrected" to the wrong data** — the dominant outcome, not an edge case — and only 13.3% (3,150, an exactly derivable count) are safely flagged uncorrectable. Zero land on the right answer by chance. This is not a bug; it is algebraic necessity: with exactly 2 syndrome equations and exactly 2 unknowns (one hypothesized error's position and magnitude), solving for that single hypothesis consumes both equations, so the decoder's post-correction re-check is guaranteed to pass — self-cancellation, not verification — whenever the initial solve finds any candidate at all. Adding more parity symbols would buy independent detection margin, at the cost of payload capacity for a guarantee an already-built mechanism provides for free.

The composition, therefore, not a substitute. RS repairs the common single-nibble-flip case for free; mirror pairing (§10) is the independent, full-payload check for anything RS cannot itself decide it got right. The two are documented as a pair everywhere they are exposed — C ABI, Python, WASM, Unity — precisely so a caller reaching for one does not mistake it for the other's guarantee.

12. Instant multi-writer convergence: a CRDT layer above the immutable pack

Sections 10 and 11 answer "is what I read intact." This section answers a different question: multiple independent writers — two devices, an offline session and a synced one — each want to write *now*, with no coordination, and later agree on one converged state. The substrate's own answer to "how do writes work" is, by design, that they don't: a `.aamt` file is immutable once packed (§3). This section does not change that. It adds a layer *above* the pack — `packages/libaamt/bindings/python/aamt_crdt.py` — that lets independent writers accumulate freely and converge deterministically, with the pack itself remaining exactly what it already was: a one-time, immutable snapshot, now of the *converged* state rather than a single writer's state.

12.1 Hybrid Logical Clock and the LWW-map

Each replica keeps a Hybrid Logical Clock, `(physical_ms, logical, replica_id)` (Kulkarni et al.'s construction, the same one CockroachDB and MongoDB use for exactly this problem), compared lexicographically in that field order. `tick()` advances a replica's own clock for a local write; `update()` folds in a remote event's clock so the receiving replica's next local write is provably causally after anything it has seen. `CrdtStore` is a map from record id to whichever write carries the largest key for that id — a Last-Writer-Wins register per id, a map of independent per-key `max` operations, which is the textbook construction for a conflict-free replicated map: `merge` is commutative, associative, and idempotent because per-key `max` over a total order is all three, and a map's merge is just that operation applied independently key by key.

12.2 What property testing found

"Textbook construction" is a claim, not a proof, and this paper's standard (§6, §11) is measurement over assertion. `examples/test_crdt.py` runs randomized trials of `merge(A,B) == merge(B,A)`, `merge(merge(A,B),C) == merge(A,merge(B,C))`, and `merge(A,A) == A` across independently generated stores. The first implementation — tiebreak on `hlc` alone, keeping whichever record was already present on an exact tie — failed commutativity on trial 2 of the first run (seed 1234, 500 trials). Traced to: two records (ids 0 and 3 in that trial) that shared an identical HLC — `(physical=4, logical=6, replica_id=0)` — and, by the test's own data encoding, identical `data` bytes, but *different* `tera` coordinates. On an exact-HLC tie, "keep whichever was already in the map" is a function of merge order, not record content: `merge(A,B)` keeps `A`'s version, `merge(B,A)` keeps `B`'s. Two records agreeing on HLC and payload bytes but disagreeing on coordinate is not a pathological input to reject — it is a legitimate, if rare, real case (colliding replica ids, a logical counter that hasn't diverged yet), and the fix is to make the tiebreak fully deterministic rather than to assume the collision away: `put()` now compares the tuple `(hlc, data, tera)` in full, so the winner on any tie is a pure function of the two candidate records' content, independent of which store `merge` started from. After the fix: 500 trials clean, then 8,000 more across four independent seeds with no failures.

Two further tests exercise cases the randomized trials don't reliably hit: a *genuine* tie — two replicas ticking a fresh clock at the identical physical millisecond, distinguishable only by `replica_id` — resolves

identically regardless of merge order; and `update()` is checked to strictly advance a receiving replica's clock past whatever remote event it just learned about, the actual property an HLC buys over a plain per-replica counter.

12.3 Convergence through a real pack

`CrdtStore.to_aamt()` packs the current converged map through the existing `AamtBuilder` — a normal call, not a new persistence path. The end-to-end test simulates the case this section exists for: two "offline" replicas each write independently (one shared id, edited on both, plus one id unique to each), merge in both orders, and pack. Both merge orders produce the identical converged store; the packed `.aamt` file, read back through the ordinary substrate reader, contains exactly the HLC-determined winner for the shared id and both unique records untouched — the convergence guarantee demonstrated surviving all the way through a real pack, not just as an in-memory dictionary comparison.

12.4 What this deliberately does not do

Last-Writer-Wins *discards* the losing write outright — there is no merge-both-versions path, by design: this is record-level replacement semantics (a whole payload wins or loses), not a text-CRDT merging concurrent intra-record edits, and the substrate has no concept of a sub-record diff to merge. A replica's logical counter is not persisted across restarts; a replica that restarts with a fresh HLC relies on its physical clock, not a remembered logical high-water mark, to stay ahead of its own prior writes — correct as long as physical time actually advances between restarts, unstated as a guarantee beyond that. And the `(hlc, data, tera)` tiebreak compares `tera` as an ordinary float tuple; nothing in this layer independently re-validates that coordinates are NaN-free before comparing them (the builder's own `clampTera4` is what is relied on upstream, same as everywhere else in this paper's pipeline).

13. Two decisions, not two function calls

WP-25 named the actual cost of wiring sections 10–12 into a live consumer: each one needs a real engineering decision, not just a call site. This section makes two of them and builds what they require.

13.1 Mirror pairing: what to serve on an unrecoverable payload mismatch

Section 10.2 stated the honest limit and stopped there: two copies with no independent checksum cannot decide which is corrupt on a pure payload disagreement. That is correct as far as it goes, but it leaves open what a caller should actually *do* — and separately, section 11 composes badly with section 10 if left as two mechanisms a caller must sequence by hand, since RS's own single-symbol "corrected" report is not, by itself, sufficient grounds for trust (the 86.7% finding is exactly about how confident a wrong answer can look). `Reader::readVerified` wires the two together and makes the decision:

1. **Primary decodes cleanly, or with an RS-repaired correction, and nothing contradicts it** (mirror absent, mirror itself unreadable, or mirror agrees): trust it. An absent or inconclusive mirror is not counter-evidence.
2. **Primary decodes, and an independently-decoded mirror positively disagrees**: this is the one case with real evidence of an unresolved conflict between two independently stored, independently decoded copies — genuinely undecidable with what this format stores, so it is refused (`AAMT_READ_UNAVAILABLE`), not guessed.
3. **Primary is missing or its RS syndrome is unresolvable outright, and the mirror decodes cleanly**: recover from the mirror — the same structural fallback `recoverRecord` already does, generalized to also cover "primary present but RS gave up."
4. **Both unreadable**: refused.

The decision, stated plainly: *counted rather than silently served* (section 10.2's own phrase) extends to its logical conclusion — a caller gets a clear "unavailable," not a coin flip dressed up as data. This is a deliberately narrower trigger for refusal than it might first seem: only a *positive*, decoded disagreement blocks trust, because that is the only case with actual evidence against the primary. Built on a new combined writer, `Builder::addMirroredWithECC`, which stores the *same* RS-encoded payload at both the primary

and the mirror leaf, so either copy can self-repair a single corrupted nibble for free and the mirror can also independently verify whatever RS decoded.

Verified with the case that matters most. `tests/roundtrip.cpp` section 7 packs 200 records with both defenses and exercises four real, on-disk-byte-edited cases: a clean read; a single corrupted nibble (RS-repaired, mirror never touched); a scrambled id (structural recovery via the mirror); and — found by exhaustively searching the same space `test_rs_gf16.cpp` already enumerates — a genuine two-nibble collision in the primary's parity nibbles (`(pos1,pos2,delta1,delta2) = (0,1,1,2)`) that RS reports as a confident, resolved repair and that is, precisely as section 11 measured, wrong. The test first confirms the danger is real — calling `aamt_decode_ecc_payload` directly on this primary reports `corrected=1` and returns the wrong bytes with no indication anything is amiss — then confirms `aamt_read_verified` catches it: the untouched mirror decodes to the true data, the two disagree, and the function refuses rather than repeats the mistake RS alone would have made silently. Exposed with full parity: C ABI (`aamt_builder_add_mirrored_ecc`, `aamt_read_verified`), Python (`AamtBuilder.add_mirrored_ecc`, `AamtSubstrate.read_verified`), WASM (exported), and Unity (`AamtNative`/`AamtSubstrate.ReadVerified` — a reader-side wrapper only, matching this platform's existing Unity scope: no managed builder wrapper exists there for any write path, this one included).

13.2 CRDT: the missing piece behind "needs an actual second writer"

Section 12 was explicit that wiring the CRDT layer into a single-writer process would be building for a hypothetical. At the time this was written, that was true — nothing in this platform had a second real writer for it to reconcile against. §13.3 changes that; this subsection's account of *why* a safe identity mechanism needed to exist first is unchanged and still the reason §13.3's second writer didn't collide with the first. "No second writer exists yet" and "nothing is missing" are different claims, and one concrete piece *was* missing: `HLC.zero(replica_id=N)` requires the caller to already have a stable, globally-unique replica identity, and nothing in `aamt_crdt.py` provided one. Left unaddressed, the first real second writer would face the exact failure mode section 12.2's own property testing found and fixed for content — except for *identity*: two independent processes each defaulting to `replica_id=1` would collide, and unlike the content collision (fixed by widening the tiebreak key), a replica identity collision is not something `merge()` can paper over — it is a modeling error upstream of the algebra.

`local_replica_id(state_dir)` closes that gap: a random 63-bit id, generated once and persisted under `state_dir/replica_id` (atomic write-then-rename, so a crash mid-write cannot leave a torn value), so every later call in the same `state_dir` — including across process restarts — returns the same id. This does not solve everything section 12.4 already flagged: the HLC's logical counter still is not persisted across restarts, only the replica identity attached to it is. What it does solve is the one piece that was a real, if latent, correctness landmine rather than a documented limitation: *when* a second writer eventually exists, it gets a safe identity by calling one function instead of inventing a scheme, the same way a machine has one hostname rather than a fresh one guessed per process. Verified in `examples/test_crdt.py`: stable across repeated calls (simulating a restart), distinct across separate installs, and confirmed actually persisted to disk rather than held only in memory.

13.3 CRDT's actual second writer: a browser tab

`aamt-crdt.js` (AAMT-Mathematics-Visualizer repository) is a JS port of `aamt_crdt.py` exact enough to matter: the same Hybrid Logical Clock algorithm, the same `(hlc, data, tera)` tiebreak (§12.2's fix, ported deliberately rather than re-derived, since re-deriving it independently would risk not reproducing the exact same fix), and the same JSON interchange format — `CrdtStore.toJson()`/`.fromJson()` on the JS side, `to_json()`/`from_json()` on the Python side. A browser tab is now a genuine second replica, not a simulated one: `localReplicaId()` persists a random 63-bit identity in `localStorage`, the browser's equivalent of `local_replica_id`'s state-dir file.

One real cross-language bug, found by testing, not assumed. A 63-bit replica id safely exceeds JavaScript's 2^{53} safe-integer range, so `aamt-crdt.js` must serialize `replica_id` as a JSON *string*. Python's `json.dumps` of a Python `int` would, left alone, emit a bare JSON *number* — the two writers' own export

formats would disagree with each other, defeating the point of a shared interchange format. Caught immediately by testing against a real browser tab (not by reasoning about the two implementations in the abstract): `to_json` now always emits `replica_id` as a string; `from_json` calls `int(...)` on whatever it receives, accepting either representation regardless of which side sends it.

Verified end to end, both directions of rigor. `aamt-crdt.js`'s own property tests (`src/aamt-crdt.test.js`, 500 randomized trials via a seeded PRNG, `node src/aamt-crdt.test.js`) prove the same three join-semilattice laws §12.2 proved for the Python side — not assumed from the port, independently re-verified. And a genuine two-process, two-language integration: a real browser tab (`preview_start` + `javascript_tool`, not a mock) wrote three records with real `Date.now()`-based HLC timestamps, exported them via `toJson()`; a real, separate Python process imported that exact JSON via `from_json()`, merged it with its own independent writes — including a deliberate id collision the desktop side's earlier HLC should lose — and packed the converged result to a real `.aamt` file. Merge order didn't matter (commutative, both directions checked); the collision resolved to the browser's later write, correctly; the packed file's content matched exactly. That exact browser-exported JSON string is pinned as a permanent regression check in both `src/aamt-crdt.test.js` and `examples/test_crdt.py`'s `test_real_cross_language_interop`, so this isn't a claim that held once in a terminal — it's a fact both test suites check on every run.

What this does not yet mean. A second writer existing doesn't make CRDT *live* in the WP-25 sense — `memory_index_bridge.py` still doesn't call any of this, and the visualizer's demo page doesn't either; nothing in the shipped product currently generates real cross-device writes to converge. What changed is narrower and real: the excuse "no second writer exists to test this against" is gone, replaced by a working one, proven to interoperate, not merely built to spec on paper.

Reference implementation: `packages/libaamt`. First production integration: `maiiam-alchemist/apps/desktop/scripts/m
Prior series: WP-20 (voxel-addressable memory), WP-21 (voxel-steered generation), WP-22 (Odu coverage map), EXP-01 (topological trit memory).